CHAPTER 12

Scaling Laws and Benchmarking

In this chapter

Introduction: The Capability That Appeared From Nowhere . . . 3
12.1 What a Scaling Law Is . . . 4
12.2 The Compute Budget: C ≈ 6ND . . . 5
12.3 Kaplan, Chinchilla, and the Correction That Reset the Industry . . . 5
12.4 Inference-Optimal Is Not Training-Optimal . . . 6
12.5 The Experiment: Eleven Models, One Power Law . . . 7
12.6 Extrapolation, and Where It Fails . . . 7
12.7 Emergence: The Same Checkpoints, Two Stories . . . 8
12.8 What Scaling Laws Do and Do Not Predict . . . 10
12.9 Benchmarks: What They Measure . . . 10
12.10 The Five Ways a Benchmark Table Misleads . . . 11
12.11 How to Read a Report That Is Done Properly . . . 12
12.12 The Only Benchmark That Decides Anything . . . 13
12.13 Code Walkthrough: Fitting a Law and Measuring a Knee . . . 13
12.14 Mini Project: The Benchmark Reality Check . . . 15
12.15 Hands-on Labs . . . 17
12.16 Lab Solutions . . . 18
12.17 Best Practices . . . 19
12.18 Common Mistakes . . . 19
12.19 Performance Considerations . . . 21
12.20 Security Considerations . . . 22
12.21 Real Production Examples . . . 22
12.22 Certification Notes . . . 23
12.23 Practice Questions . . . 24
12.24 Answers and Explanations . . . 25
12.25 Ten Practical Exercises . . . 26
12.26 Five Challenge Labs . . . 27
12.27 Interview Questions . . . 27
12.28 Summary . . . 28

12.29 Glossary 29
12.30 Further Reading 30

Part 2 of 13 · Chapter 12 of 64. Next: Chapter 13 — Reasoning Models and Test-Time Compute.

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. State what a **scaling law** is, fit one from data, and say what it predicts and what it does not.
2. Use the compute identity **C ≈ 6ND** to reason about training budgets, and explain the **Chinchilla** correction and why it changed the industry.
3. Explain why modern models are deliberately **over-trained** relative to compute-optimal, and connect that to Chapter 11's inference economics.
4. Demonstrate that **"emergent" capability is frequently a property of the metric**, not of the model, and quantify the effect.
5. Explain why an **all-or-nothing metric** manufactures a knee out of a smooth curve, and compute how narrow that knee becomes as answers gain parts.
6. Name the **five ways a benchmark table misleads** — contamination, saturation, scaffolding, prompt sensitivity, and missing intervals — and check for each.
7. Compute the **sample size** needed to establish a reported gap, and say when a published difference has not been established.
8. Design the only benchmark that decides anything: **your task, your distribution, your metric**.

NOTE

Prerequisites

Chapter 8 is the backbone of the second half of this chapter — Wilson intervals, evaluation-set sizing, and the rule that a difference whose intervals overlap has not been established. **Chapter 9**, section 9.9, which claimed that a 70B model trained on 15 trillion tokens beats an identical 70B trained on 300 billion; this chapter is where that claim is explained. **Chapter 10** for the arithmetic experiment whose machinery we reuse across eleven model sizes, and for the difference between limits of degree and limits of kind. **Chapter 11** for the cost of inference, which is what makes training-optimal and deployment-optimal different questions.

Chapter 10 ended by asking whether a limit is a matter of degree or of kind, and said that waiting for a better model is a legitimate plan for the first kind. This chapter is how you estimate what waiting will buy — and how to tell a real capability gain from a chart that has been drawn flatteringly.

Introduction: The Capability That Appeared From Nowhere

A team tracks an internal benchmark across a family of models they are training. For four successive sizes, exact-match accuracy is 2.5%, 8.4%, 13.1% — and then, at the next size, **62.1%**.

The parameter count went up by a factor of 1.4. The score went up by a factor of 4.7. Somebody uses the word *threshold*. A roadmap is drawn on the assumption that the next size up will unlock a further capability, because that is what the chart plainly shows: a phase transition, a switch being thrown, something that was not there and now is.

Here is the same experiment, with a second metric printed beside the first:

```
    params     loss  per-digit    exact
```

```
  7,238   0.4442     0.8466   0.0843
 11,590   0.4709     0.8226   0.1308
 16,454   0.1667     0.9395   0.6206
 27,718   0.0863     0.9860   0.9047
```

Across the interval where exact-match leapt from 0.13 to 0.62, per-digit accuracy moved from 0.82 to 0.94, and loss fell from 0.47 to 0.17. Both of those are ordinary steps on a smooth curve. Neither shows anything resembling a transition.

Nothing discontinuous happened inside the models. The task requires seven digits and an answer counts only if **all seven** are right. Raising a smooth curve to the seventh power manufactures a knee. The threshold was in the metric.

This chapter has two halves and they are the same argument. The first half is about **scaling laws**: the empirical regularities that let you predict what more compute buys, where they came from, what they get right, and — using eleven models we trained for this chapter — where extrapolating from them goes wrong. The second half is about **benchmarks**: how to read a published comparison without being misled by it, and how to build the only evaluation that decides anything, which is the one on your own task.

The connective tissue is a discipline you already have from Chapter 8. **A number without an interval is not a finding, and a curve without a stated metric is not a capability.** Everything below is that sentence, applied to the two places the industry most reliably forgets it.

12.1 What a Scaling Law Is

DEFINITION

Scaling law

A **scaling law** is an empirical relationship — almost always a power law — between the resources spent training a model and the loss it achieves. In the general form:

```
loss  ≈  a · X^(−b)  +  c
```

where `X` is parameters, training tokens or total compute; `b` is the exponent that governs how fast loss falls; and `c` is an irreducible floor representing the entropy of the data itself, which no model can beat.

Plotted on log-log axes, a power law is a **straight line**. That is what makes scaling laws useful: a straight line can be extended.

Three things follow from the shape and are worth internalising before any of the history.

Returns diminish, predictably. A power law with `b ≈ 0.5` means quadrupling the resource halves the excess loss. Every increment costs more for less, and the rate at which it does so is measurable rather than a matter of opinion.

There is a floor. The `c` term is the part of the loss that comes from genuine unpredictability in the data. English text has irreducible entropy; no amount of compute predicts a coin flip. As models approach `c`, the power-law term stops mattering and further scale buys almost nothing on that metric.

The relationship is empirical, not derived. Nobody proved these laws from the architecture. They were fitted to observations, they have held remarkably well across many orders of magnitude, and they

could stop holding. Treating them as physics rather than as a very good regression is the commonest error in scaling discourse.

12.2 The Compute Budget: C ≈ 6ND

Before the laws themselves, the identity that makes them actionable.

Training compute is approximately:

```
C  ≈  6 · N · D          C = FLOPs, N = parameters, D = training tokens
```

The 6 comes from the training step: a forward pass costs roughly `2N` floating-point operations per token, because each parameter participates in one multiply and one add; the backward pass costs about twice that; so a full step is about `6N` per token, and `D` tokens gives `6ND`.

This is worth carrying because it converts an abstract budget into a design decision. **For a fixed compute budget, `N` and `D` trade against each other directly.** You can train a big model on little data or a small model on lots of data, and the two cost the same. The entire scaling-law literature is about which choice is better.

12.3 Kaplan, Chinchilla, and the Correction That Reset the Industry

In 2020, Kaplan and colleagues at OpenAI published the first systematic study of transformer scaling. They found clean power laws in parameters, data and compute, and — the consequential part — concluded that for a fixed compute budget you should spend most of the increase on **parameters**, training a large model on comparatively little data.

The field followed that advice. Models grew fast and training sets did not grow proportionally.

In 2022, Hoffmann and colleagues at DeepMind revisited the question with a more careful experimental design and reached a different answer. Their conclusion — universally known as **Chinchilla** — was that parameters and data should be scaled **roughly in proportion**, and that the compute-optimal ratio is in the region of **20 training tokens per parameter**. By that standard, the large models of the preceding two years were badly under-trained: they had too many parameters for the data they had seen.

The demonstration was direct and is the reason the result was accepted so quickly. Chinchilla, at 70 billion parameters trained on 1.4 trillion tokens, outperformed Gopher, at 280 billion parameters trained on 300 billion tokens, across a wide range of evaluations — **at the same training compute**. A model one quarter the size beat one four times larger, because the compute had been allocated differently.

KEY TAKEAWAYS

Why Chapter 9's claim is true

Chapter 9, section 9.9 asserted that a 70B model trained on 15 trillion tokens beats a 70B model trained on 300 billion by a margin no architectural change could produce. This is why.

Parameter count sets capacity. **Training tokens determine how much of that capacity gets used.** Two models with identical dimensions and different data budgets are not similar models with a small difference; they are a well-trained model and an under-trained one, and the gap is large.

This is the deepest reason a parameter count does not predict quality — and the deepest reason that Claude's undisclosed parameter counts are not the loss they appear to be. Even if you knew them, the number you actually needed was the training budget, and that is not published either.

12.4 Inference-Optimal Is Not Training-Optimal

Chinchilla answers a specific question: given a fixed compute budget **for training**, what is the lowest loss you can reach? For a lab publishing a research result, that is the right question. For anyone who intends to *serve* the model, it is the wrong one.

Chapter 11 explained why. Training is paid once. Inference is paid on every request, forever, and its cost scales with the model's size — Chapter 9's decode arithmetic says a 70B model at bf16 must stream 131.4 GiB per generated token. A model that is half the size costs half as much to serve for its entire commercial life.

So the deployment-oriented question is different: given that I will serve this model billions of times, what is the **smallest** model that reaches the quality I need, and how much extra training will it take to get there? The answer is to take a smaller model and train it far past the compute-optimal point — spending more on training than Chinchilla would advise, in exchange for permanently cheaper inference.

The published open-weight models make the shift visible. Chapter 9's Llama 3.1 figures, against a Chinchilla-optimal ratio of about 20 tokens per parameter:

Model	Parameters	Training tokens	Tokens per parameter	Versus Chinchilla
Chinchilla-optimal 70B	70.5B	~1.4T	~20	1×
Llama 3.1 70B	70.55B	~15T	~213	~11× over-trained
Llama 3.1 8B	8.03B	~15T	~1,868	~93× over-trained

The 8B model saw roughly ninety times more data per parameter than the compute-optimal recipe calls for. That is not a mistake; it is a deliberate purchase of inference efficiency with training compute, and it is why small modern models are so much better than small models of a few years ago at the same size.

Two consequences for an architect. First, **a small model today is not what a small model used to be**, so intuitions about tier gaps age badly and must be re-measured — which is Chapter 15's job. Second, this is the economic reason a family like Claude's exists at several tiers at all: each tier is a different point on the training-cost-versus-serving-cost curve, and Chapter 11's rate card is the visible shadow of that curve.

12.5 The Experiment: Eleven Models, One Power Law

Scaling laws are usually met as a plot in someone else's paper. Here is one fitted from scratch, on models trained for this chapter.

We took the addition network of Chapter 10 and trained it at **eleven sizes**, from 1,258 to 73,798 parameters — a 59-fold range — for the same number of steps, on the same data, evaluated on the same held-out set. Three numbers were recorded for each: mean cross-entropy loss, per-digit accuracy, and exact-match accuracy.

```
   params     loss  per-digit    exact     pd^7
    1,258   1.1747     0.5460   0.0008   0.0145
    1,670   0.8961     0.6514   0.0060   0.0498
    2,518   0.9160     0.6208   0.0054   0.0356
    3,398   0.7298     0.7367   0.0249   0.1178
    5,254   0.7335     0.7245   0.0255   0.1048
    7,238   0.4442     0.8466   0.0843   0.3117
   11,590   0.4709     0.8226   0.1308   0.2550
   16,454   0.1667     0.9395   0.6206   0.6462
   27,718   0.0863     0.9860   0.9047   0.9058
   41,030   0.0621     0.9881   0.9202   0.9198
   73,798   0.0194     0.9977   0.9844   0.9837
```

Fitting a power law to loss against parameters:

```
fit on all 11 models:  loss = 1673.073 * N^-0.957     R^2 = 0.9067
per-digit error:      err  = 5527.645 * N^-1.225     R^2 = 0.8776
```

Two clean power laws over a 59× range. Note the honest R^2 values: 0.91 and 0.88 are real trends and they are not the 0.99-plus fits that published scaling-law work reports, because we ran **one seed per size** over a narrow range. Two points on the table are visibly out of order — 2,518 parameters is slightly worse than 1,670, and 11,590 slightly worse than 7,238 — which is exactly the run-to-run variance you would average away with more seeds. Reporting the fit without that caveat would be the same error this chapter is about to accuse benchmark tables of.

12.6 Extrapolation, and Where It Fails

The commercial appeal of a scaling law is extrapolation: fit on the models you can afford, predict the model you cannot. So we tested it. Fitting on the seven smallest models and predicting the four largest:

```
fitted on the 7 smallest (1,258-11,590 params):  loss = 22.225 * N^-0.417
   params    actual  predicted    ratio
   16,454    0.1667     0.3861     2.32x
   27,718    0.0863     0.3105     3.60x
   41,030    0.0621     0.2636     4.24x
   73,798    0.0194     0.2063    10.61x
```

The extrapolation is wrong, and it is wrong in the optimistic direction — it **under-predicted** how good the larger models would be, by up to a factor of ten in loss.

This is a genuinely useful failure, and it says three things.

A fit over a 9× range does not license prediction over a 59× one. Real scaling-law studies fit across many orders of magnitude, with multiple seeds per point and careful control of the learning-rate schedule. Ours is a toy, and the toy's extrapolation failed in precisely the way that methodology exists to prevent.

The exponent is the fragile part. Our seven-model fit gave `b = 0.417`; the full eleven-model fit gave `b = 0.957`. The exponent is a slope, and slopes estimated from a short, noisy segment are unreliable. Since the exponent is what compounds over the extrapolation, a small error in it becomes a large error in the prediction.

Scaling laws predict a trend line, not a model. Even a good fit tells you the expected loss of an average model at a given size. It does not tell you that *your* training run will land on the line, and it says nothing at all about the specific capability you care about — a point section 12.8 makes properly.

12.7 Emergence: The Same Checkpoints, Two Stories

Now the chapter's central result, and the reason its introduction is not a hypothetical.

Emergent capability is the widely reported phenomenon in which a model shows near-zero performance on a task up to some scale and then, over a comparatively small increase, becomes competent. It has been taken as evidence that scaling produces qualitative jumps — that something appears which was not there before.

We measured how wide the transition is for each of our three metrics, defined as the parameter range over which the metric crosses from 10% to 90% of its total span:

```
metric                 10% of span  90% of span    width
loss (lower better)          1,670       27,718    16.6x
per-digit accuracy           1,670       27,718    16.6x
exact-match accuracy        11,590       27,718     2.4x
```

Loss and per-digit accuracy take a 16.6× parameter range to cross. Exact-match takes 2.4×. Those are the same eleven checkpoints. The models did not change between the rows of that table; only the metric did.

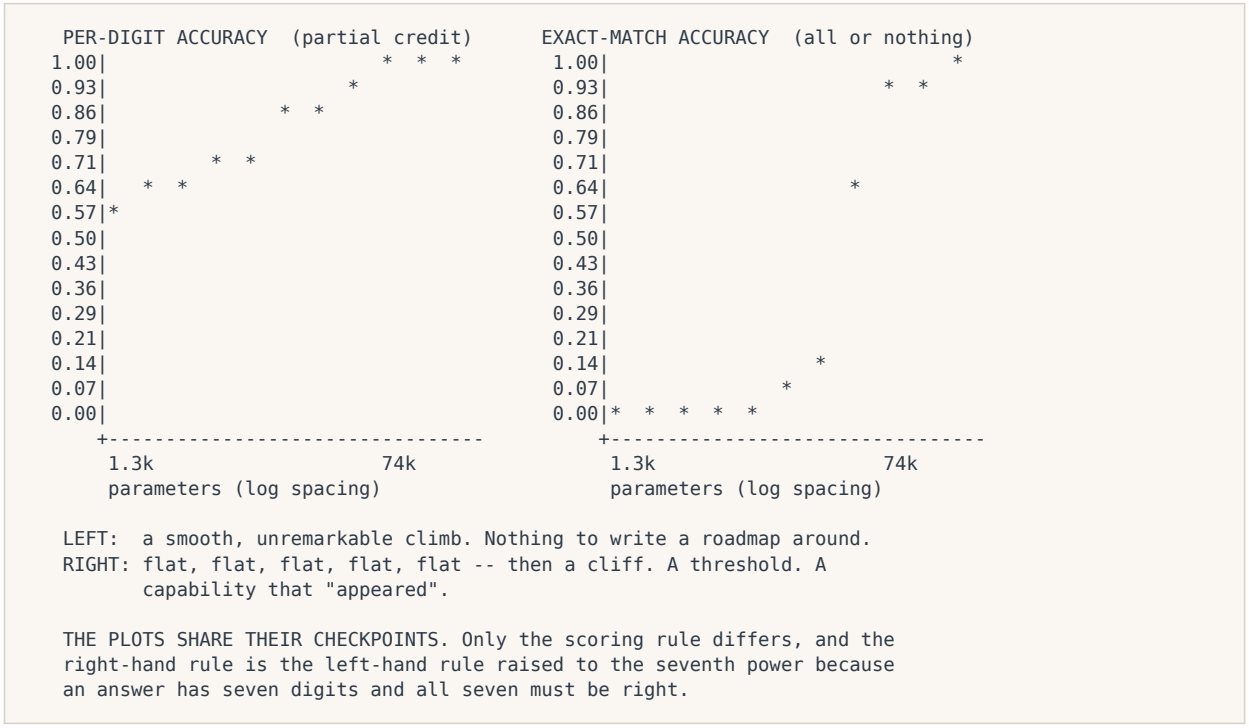


Figure 12.1 — The same eleven models, scored two ways.

The mechanism is the last column of section 12.5's table. An answer here has seven digits and counts only if all seven are right, so exact-match is approximately per-digit accuracy raised to the seventh power. The approximation is measurably good where it should be:

```
above 0.9 per-digit, mean |pd^7 - exact| = 0.0069
below 0.9 per-digit, mean |pd^7 - exact| = 0.0874
```

Above 0.9 per-digit accuracy the prediction is accurate to well under one percentage point. Below it the approximation overestimates, because digit errors are correlated rather than independent — a model that has not learned a column gets that column wrong on nearly every example. We report both, because the second number is the limit of the explanation and hiding it would be the same sin as reporting a benchmark without its failure cases.

Generalising the mechanism away from digits: for a task whose answer has `k` parts that must all be right, here is the per-part accuracy range that takes the aggregate score from 10% to 90%.

```
parts   per-part range for 10% -> 90%       span
    1   0.1000 -> 0.9000                  0.8000
    3   0.4642 -> 0.9655                  0.5013
    7   0.7197 -> 0.9851                  0.2654
   20   0.8913 -> 0.9947                  0.1035
   50   0.9550 -> 0.9979                  0.0429
```

At fifty parts — an agentic trajectory, a long proof, a multi-file code change — the entire visible transition from "useless" to "works" occupies a per-part accuracy band from 0.955 to 0.998. Any underlying capability curve, however smooth, crossing that band will look like a switch being thrown.

VISUAL ANALOGY

Emergence is a high-jump bar, not a growth spurt

Measure a child's height monthly and you get a smooth curve. Measure instead whether they can clear a bar set at 1.4 metres and you get a step function: no, no, no, no, yes. Nobody watching the second measurement would conclude that the child's growth was discontinuous. The discontinuity is the bar.

Almost every headline capability benchmark is a bar. *Did it solve the problem? Did the code compile and pass the tests? Was the answer exactly right?* All-or-nothing, all of them, and all of them therefore capable of turning smooth improvement into an apparent phase change.

Where the analogy breaks. A bar has one clearly-stated height, whereas a benchmark's implicit "bar" is the conjunction of every part of the task that has to go right at once, and that conjunction is usually undocumented. It is also unfair to the emergence literature to imply the effect is *only* metric choice: there is genuine debate, some capabilities do improve faster than a simple power law over some ranges, and in-context learning itself was not predicted by anyone. The defensible claim is narrower and still very strong: **a sharp curve is not evidence of a qualitative change until you have checked the metric.** Plot a continuous score on the same checkpoints. If the sharpness survives, you have found something. Usually it does not.

12.8 What Scaling Laws Do and Do Not Predict

Scaling laws predict well	Scaling laws predict badly or not at all
Loss on the training distribution	Whether a specific capability will work
The aggregate trend across many tasks	Performance on your particular task
Roughly, where diminishing returns set in	When an all-or-nothing metric will move
Whether more compute is worth spending	Instruction-following and format reliability
The relative benefit of parameters versus data	Refusal behaviour, calibration and safety properties

The right-hand column is not a list of hard research problems. It is a list of things that are **not loss**. A scaling law is a statement about average next-token prediction over a broad distribution. Your task is one thin slice of that distribution measured with a metric of your own choosing, and the relationship between the two is loose.

This is Chapter 10's question — degree or kind? — with a quantitative handle on the *degree* branch. Waiting for a better model is a legitimate plan when the failure is one of degree, and scaling laws tell you roughly how much waiting buys on average. They do not tell you whether it buys enough for you. Only your own evaluation does, which is where the rest of this chapter goes.

12.9 Benchmarks: What They Measure

DEFINITION

Benchmark, contamination, saturation

A **benchmark** is a fixed set of tasks with a scoring procedure, used to compare models. Its value comes entirely from being fixed, and that is also the source of every problem it has.

Contamination is the presence of benchmark items, or close paraphrases, in a model's training data. A contaminated benchmark measures memorisation and reports it as capability.

Saturation is what happens when scores approach the ceiling. Once several models score above 95%, the benchmark no longer distinguishes them, and the remaining few points are frequently the benchmark's own label errors rather than the models' failures.

Benchmarks are not useless. They are how the field measures progress at all, they catch catastrophic regressions, and a model that scores badly across the board is genuinely worse. What they cannot do is tell you which model to deploy, because they are not measuring your task, your distribution, your prompt or your metric.

12.10 The Five Ways a Benchmark Table Misleads

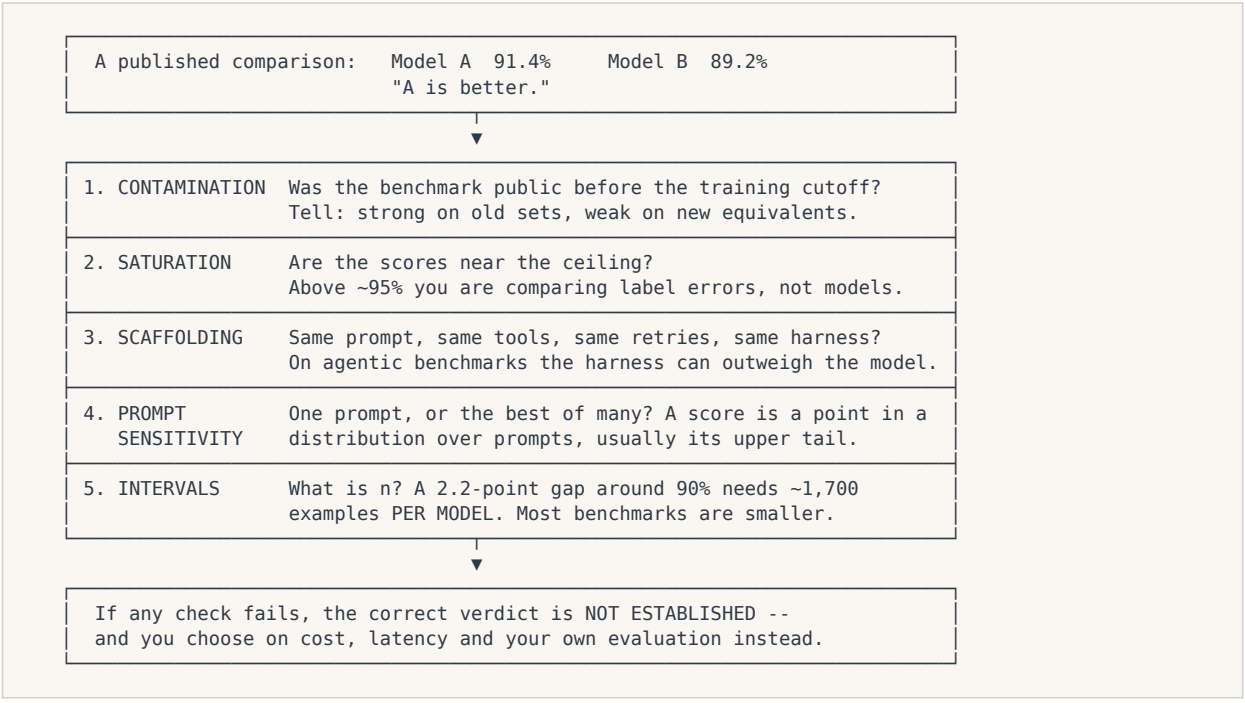


Figure 12.2 — Five checks to run before a benchmark table changes a decision.

1. Contamination. The internet contains benchmark items and their solutions. Any benchmark published before a model's training cutoff is potentially in its training data. Symptoms: implausibly strong performance on old benchmarks alongside weak performance on structurally identical new ones; and a large gap between a public benchmark and a private variant of the same task.

2. Saturation. A comparison of 96.1% against 95.8% is usually a comparison of two models' handling of the benchmark's mislabelled items. When scores cluster near the ceiling, the benchmark has stopped measuring the models and started measuring itself.

3. Scaffolding differences. A published score is a score for a *system*: the model plus a prompt, a retry policy, a tool set, sometimes a whole agent harness. Two rows in a table may differ more in scaffolding than in model. This matters most on agentic benchmarks, where the harness can be worth more than the model choice — and it is the reason Chapter 10's compensating architecture is often the larger effect.

4. Prompt sensitivity. The same model on the same benchmark scores differently under different prompts, sometimes by several points. A published number is one point in a distribution over prompts, and it is usually the best one the authors found.

5. No intervals. The one Chapter 8 already prepared you for, and the most common. A benchmark of a few hundred items produces a score with a confidence interval several points wide, and the interval is almost never printed.

That last point deserves numbers, because the numbers are stark. Using the two-proportion arithmetic from this chapter's harness:

```
gap to establish, around a 90% rate    examples needed PER MODEL
5 points                                                   277
2 points                                                 1,729
1 point                                                  6,915
```

A two-point difference around 90% needs about 1,700 examples per model to establish. Most benchmarks are smaller than that. So the majority of two-point gaps in published tables are consistent with no difference at all — and a model-selection document that cites one as a reason is citing noise.

12.11 How to Read a Report That Is Done Properly

It is fair to note that the better published reporting is considerably more careful than the tables that circulate on social media, and it is worth knowing what good looks like.

Anthropic publishes a **system card** for each model — the Claude Opus 5 system card is dated 24 July 2026 and the Claude Sonnet 5 system card 30 June 2026 — covering capability evaluations, safety testing and, for the current generation, external red-teaming by third parties including the UK AI Security Institute. According to the published material at the time of writing, the Opus 5 card reports a range of named evaluations including Frontier-Bench and GDPval-AA for coding and knowledge work, and ExploitBench, OSS-Fuzz, CyScenarioBench and ExploitGym for cyber capability.

The detail worth copying is how the aggregate is stated. The Opus 5 capability index is reported as **a point estimate of 162.1 with a 95% confidence interval of 158.0 to 167.3, over n = 40 benchmarks**. †

An interval and a sample size. That is exactly what section 12.10's fifth failure is the absence of, and it is what you should demand of any comparison — including your own. When you report your evaluation to your own team, report it that way.

Three habits for reading any table:

Find `n` before you read the score. If it is not stated, the score's precision is unknown and you should treat differences under about five points as unestablished.

Ask what the scaffolding was. Especially on agentic benchmarks. "Model A beats model B" and "harness A beats harness B" look identical in a bar chart.

Ask whether the benchmark resembles your task. Not thematically — mechanically. Same input length, same output format, same tolerance for partial correctness, same failure cost.

12.12 The Only Benchmark That Decides Anything

Everything above is preparation for a short and unglamorous conclusion: **build your own evaluation set, and select on it.**

Chapter 8 gave the methodology and the sample sizes. This chapter adds four requirements specific to model selection.

Draw from your real distribution, including the awkward tail. Chapter 10's edge-of-distribution lesson applies directly: behaviour at the edges is discontinuous rather than degraded, and an evaluation sampled from the same place as your examples cannot detect that by construction.

Choose your metric before you look at any scores, and choose it to match the decision the output feeds. If a partial answer is worthless to your product, measure exact-match and accept that your curve will look sharp. If partial credit is genuinely useful, measure it and get a smoother, more informative signal. Reporting both, as section 12.7 did, is better than either — and it is the single cheapest protection against the emergence illusion.

Size it for the difference you need to detect. If the models you are choosing between are within a couple of points, you need on the order of a thousand examples per model, and if you cannot afford that then you cannot resolve that difference — which is a legitimate finding. Choose on cost and latency instead, and say so.

Keep it private and keep it fresh. A benchmark you publish is a benchmark that will eventually be trained on. A benchmark you never refresh will saturate against your own prompt improvements.

This is not a counsel of perfection. A hundred examples from your own traffic, scored with a metric you chose deliberately, will tell you more about which model to deploy than any public leaderboard — provided you report the interval alongside the number and are willing to say "not established" when it is.

12.13 Code Walkthrough: Fitting a Law and Measuring a Knee

Three small pieces of code produced everything in sections 12.5 to 12.7. Each is short, and each contains one decision that matters more than the rest of it.

The sweep. Eleven models, and the important lines are the ones that hold everything else constant:

```
SIZES = [6, 8, 12, 16, 24, 32, 48, 64, 96, 128, 192, 288]
STEPS = 2500
def train_one(h: int, steps: int = STEPS, seed: int = 11) -> dict:
    rng = np.random.default_rng(seed)
    net = MLP(h=h, seed=seed)
    X, Y, _, _ = make(120_000, 0, TRAIN_MAX, rng)
    for s in range(steps):
        i = rng.integers(0, X.shape[0], 512)
```

```
        net.step(X[i], Y[i], 2e-3 if s < steps * 0.75 else 4e-4)
    # held-out evaluation, identical set for every model
    erng = np.random.default_rng(4242)
    Xe, Ye, _, _ = make(20_000, 0, TRAIN_MAX, erng)
```

The evaluation RNG is seeded **separately and identically** for every size. If the eval set varied between models, the comparison would be partly measuring the eval set, and a scaling curve assembled that way is noise with a trend drawn through it. There is a test asserting this, which is unusual for a seeded RNG and is there because the failure would be silent.

`SIZES` lists twelve entries and the results file contains eleven. The largest did not finish within the session's time budget, and rather than quietly trimming the list we left it — the analysis reads whatever has completed. A sweep that can be extended without invalidating what came before is worth slightly untidy code.

Three numbers come back per model, and choosing them was the whole experiment:

```
    loss = float(-np.log(np.take_along_axis(p, tgt, -1).clip(1e-12)).mean())
    pred = logits.argmax(-1)
    return {
        "loss": loss,
        "per_digit": float((pred == Ye).mean()),
        "exact": float((pred == Ye).all(1).mean()),
    }
```

`.mean()` and `.all(1).mean()` differ by five characters and produce the two halves of Figure 12.1. Chapter 10 made this point about a single model; here it produces two different accounts of an entire research programme.

The fit. A power law in log-log space is a straight line, so fitting one is linear regression on logarithms:

```
def fit_power(n, y):
    """log y = log a - b log n. Returns (a, b, r2)."""
    ln, ly = np.log(n), np.log(y)
    b, loga = np.polyfit(ln, ly, 1)
    pred = loga + b * ln
    ss_res = float(((ly - pred) ** 2).sum())
    ss_tot = float(((ly - ly.mean()) ** 2).sum())
    return float(np.exp(loga)), float(-b), 1 - ss_res / ss_tot
```

Returning `r2` alongside the coefficients is not decoration. A power-law fit will return *some* exponent for any data you hand it, including data with no relationship at all, and the exponent will look authoritative because it has three decimal places. The R^2 is what tells you whether the straight line was there before you drew it. Ours were 0.907 and 0.878 — real, and not good enough to extrapolate from, which section 12.6 then demonstrated rather than asserted.

The knee measurement. This is the piece that turns an argument into a number:

```
def transition_width(n, y, ascending=True):
    """Parameter range over which a metric crosses 10% to 90% of its span.
    A wide transition means the capability arrives gradually. A narrow one
    looks like a phase change. Same models either way.
    """
    y = np.asarray(y, dtype=float)
```

```
    lo, hi = y.min(), y.max()
    t10, t90 = lo + 0.10 * (hi - lo), lo + 0.90 * (hi - lo)
```

Normalising to each metric's **own** span is the key choice. Loss runs from 1.17 to 0.02; exact-match from 0.0008 to 0.98. Comparing raw movements would compare units. Comparing the *parameter range each metric needs to cross its own span* asks the only question that matters — how abrupt does this look? — and makes the answer comparable across metrics measured in different things.

An earlier version of this function measured the largest single-step ratio instead. It gave exact-match a "biggest jump" of 8.07× located at the very bottom of the curve, where the score moves from 0.0008 to 0.006 and the ratio is enormous while the movement is negligible. The metric was measuring the smallness of the denominator. It was replaced because it produced a confident number that pointed at the wrong interval — which is, unhelpfully for the author's dignity, an excellent illustration of this chapter's subject.

12.14 Mini Project: The Benchmark Reality Check

What you are building. Two things that belong together: a scaling sweep that fits a law and measures how abrupt each metric looks, and a small statistical toolkit that tells you whether a reported gap between two models is a finding or a coincidence.

Why it is worth building. Because both halves answer questions that get answered by assertion today. "Will a bigger model fix this?" is currently answered with a shrug; a fitted curve on your own task answers it with a slope. "Is model A better than model B?" is currently answered by reading a table; the two-proportion interval answers it with an established-or-not.

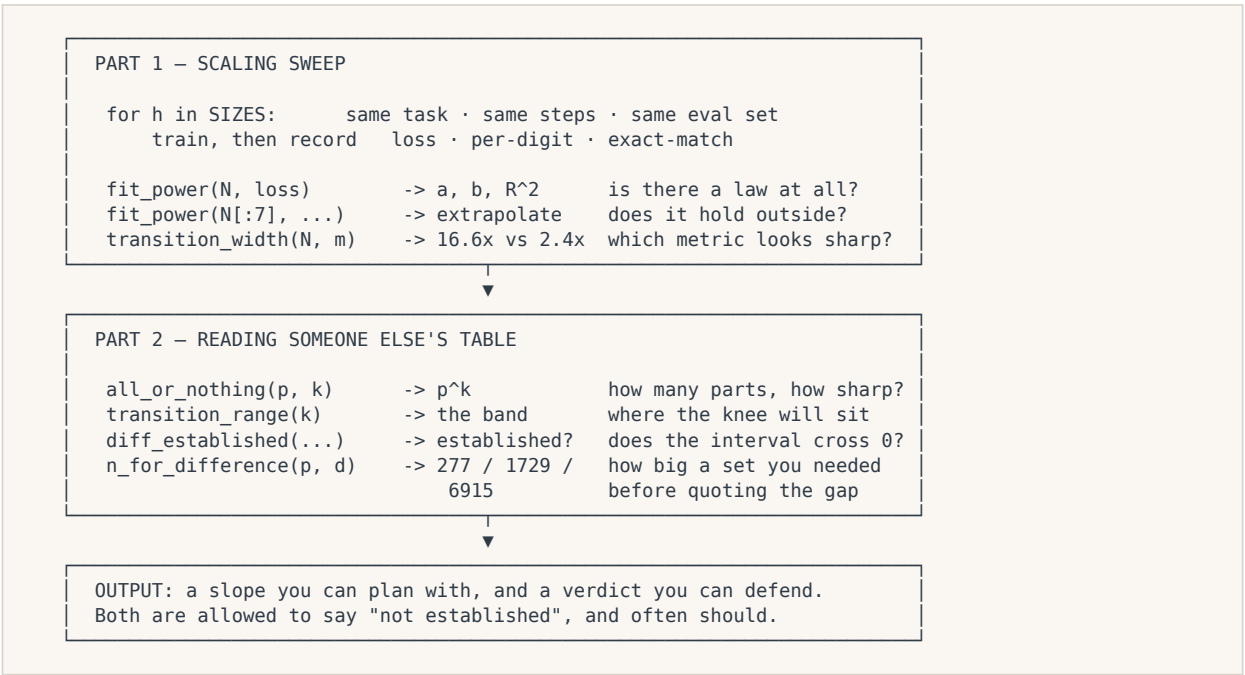


Figure 12.3 — The benchmark reality check.

Folder structure.

```
ch12/
```

```
  arith.py           the task and the network, carried over from Chapter 10
  scaling.py         the sweep; resumable, appends to scaling.json
  analyse.py         power-law fit, extrapolation test, transition widths
  metrics.py         all-or-nothing distortion and two-proportion intervals
  scaling.json       11 trained models, the chapter's raw data
  test_scaling.py    9 tests asserting the published findings
  test_metrics.py    6 tests for the statistical tools
```

The second half is where the daily value is. `metrics.py` is sixty lines and answers the question every model-selection document gets wrong:

```
def diff_established(k1: int, n1: int, k2: int, n2: int, z: float = 1.96):
    """Two-proportion difference with a normal-approximation interval.
    Returns (difference, lo, hi, established). 'Established' means the
    interval excludes zero -- which most published comparisons do not.
    """
    p1, p2 = k1 / n1, k2 / n2
    d = p1 - p2
    se = np.sqrt(p1 * (1 - p1) / n1 + p2 * (1 - p2) / n2)
    lo, hi = d - z * se, d + z * se
    return d, float(lo), float(hi), bool(lo > 0 or hi < 0)
```

And the planning tool that stops the problem happening in the first place:

```
def n_for_difference(p: float, delta: float, z: float = 1.96) -> int:
    """Examples per arm needed to establish a difference of `delta`
    around a rate of `p`. The number nobody checks before quoting one."""
    return int(np.ceil(2 * z * z * p * (1 - p) / (delta * delta)))
```

```
  gap to establish, around a 90% rate    examples needed PER MODEL
  5 points                                                   277
  2 points                                                 1,729
  1 point                                                  6,915
```

The `delta²` in the denominator is why: halving the gap you want to resolve **quadruples** the examples you need. A test pins the relationship, because it is the fact that makes people give up on resolving small differences and choose on cost instead — which is usually the right decision anyway.

The distortion tool generalises Figure 12.1 beyond our seven digits:

```
def transition_range(parts: int, lo: float = 0.10, hi: float = 0.90):
    """Per-part accuracy needed to move an all-or-nothing score from lo
    to hi. A narrow range means the metric looks like a switch."""
    return lo ** (1 / parts), hi ** (1 / parts)
```

```
  parts   per-part range for 10% -> 90%       span
      1   0.1000 -> 0.9000                  0.8000
      7   0.7197 -> 0.9851                  0.2654
     50   0.9550 -> 0.9979                  0.0429
```

Point this at your own task before you interpret a jump. If your success criterion is "the agent completed all fifty steps", any smooth underlying improvement crossing the 0.955-to-0.998 band will present as a breakthrough.

Testing. Fifteen tests. Several assert findings, so that a reader who reproduces the chapter learns whether they reproduced it:

```
def test_exact_match_transitions_far_faster_than_per_digit():
    """The headline result. Same checkpoints, two metrics, and only one
    of them looks like a phase transition."""
    _, _, w_pd = transition_width(N, PD, True)
    _, _, w_ex = transition_width(N, EX, True)
    assert w_ex < w_pd / 4
```

Two are negative results, kept deliberately:

```
def test_extrapolating_from_a_narrow_range_overestimates_loss():
    """A negative result worth keeping. Fitted on the 7 smallest models,
    the law over-predicts loss at the largest by a wide margin. Scaling
    laws need orders of magnitude and many seeds, not a 9x span."""
    a, b, _ = fit_power(N[:7], LOSS[:7])
    assert a * N[-1] ** -b / LOSS[-1] > 3.0
def test_the_approximation_degrades_where_errors_correlate():
    """Honest limit: below 0.9 per-digit the digits fail together, so the
    independence assumption overestimates exact-match."""
    lo = [(p, e) for p, e in zip(PD, EX) if p <= 0.9]
    assert np.mean([p ** DIGITS - e for p, e in lo]) > 0.02
```

And one guards the experimental design itself:

```
def test_the_evaluation_set_is_identical_for_every_model():
    """Otherwise the comparison measures the eval set, not the models."""
```

```
$ cd ch12 && python3 -m pytest -q
...............                                                      [100%]
15 passed in 0.45s
```

Deployment. `metrics.py` belongs in the same repository as Chapter 11's cost model and Chapter 10's ledger, imported by whatever produces your model-comparison reports. The single highest-value integration is to make your evaluation harness print `established` or `not established` beside every comparison, and to refuse to print a bare percentage without `n`. It takes an hour and it permanently changes the quality of the arguments your team has.

Evaluation. Three criteria, in order.

1. **Does the harness ever say "not established"?** If it never does, either your evaluation sets are unusually large or the check is not wired up. In practice most real comparisons on a few hundred examples are unresolved, and a tool that never says so is not being consulted.
2. **Does plotting a continuous metric alongside your headline one change any interpretation?** Run it on a capability you believe "emerged". If the continuous curve is smooth, you have saved yourself a roadmap.
3. **Does a fitted slope on your own task change a waiting decision?** The point of the scaling half is to convert "let's see what the next model brings" into an estimate with an error bar, including the estimate that it will not be enough.

12.15 Hands-on Labs

Lab 12.1 — Reproduce the sweep. Run `scaling.py` until at least ten sizes have completed, then `analyse.py`. *Success criterion: you obtain a power-law exponent above 0.5 with R^2 above 0.85, and transition widths differing by at least 4× between exact-match and per-digit.*

Lab 12.2 — Break the emergence. Change the task to three-digit addition so answers have four digits rather than seven, and re-run. *Success criterion: the exact-match transition widens measurably, and you can predict the direction of the change before running it.*

Lab 12.3 — Add seeds. Train three seeds at each of five sizes and re-fit. *Success criterion: a higher R^2 than the single-seed fit, and a statement of how much of the original scatter was run-to-run variance rather than size.*

Lab 12.4 — Audit a published table. Take a real benchmark comparison and run Figure 12.2's five checks. *Success criterion: for at least one row, you can state that the difference is not established, with the sample size that would have been needed.*

Lab 12.5 — Size your own selection study. For a model choice you actually face, decide the smallest difference that would change your decision and compute the examples needed. *Success criterion: a number, and an honest decision about whether to run it or to choose on cost instead.*

12.16 Lab Solutions

12.1. Expect an exponent near 0.9 to 1.0 for loss against parameters and R^2 around 0.9. Two sizes will likely land out of order; that is single-seed variance, not a broken sweep. Transition widths should come out near 16× for per-digit and near 2.4× for exact-match. If your exact-match width is not far narrower than your per-digit width, check that you are using `.all(1)` and not `.mean()`.

12.2. With four output digits instead of seven, exact-match is per-digit accuracy to the fourth power rather than the seventh, so the knee is wider and less dramatic. From `transition_range`, the per-part band that carries an all-or-nothing score from 10% to 90% widens from 0.720–0.985 at seven parts to about 0.562–0.974 at four. The prediction is directional and confident, and it is the whole point: **the sharpness of a capability curve is a property of how many things have to go right at once, and you can compute it in advance.**

12.3. Three seeds per size typically lifts R^2 into the high 0.90s and removes the out-of-order points. The instructive part is the size of the scatter you eliminate: comparing the spread across seeds at one size against the step between adjacent sizes tells you how many of your "size effects" were seed effects. Published scaling work does this as a matter of course, and it is the difference between our 0.907 and their 0.99-plus.

12.4. The check that most often fails is the fifth. Take any two rows differing by two points, find the benchmark's item count, and compare against the roughly 1,700 per model that a two-point gap at 90% requires. Most standard benchmarks have a few hundred to a couple of thousand items, so two-point gaps are routinely unresolved. The second most common failure is scaffolding on agentic benchmarks, where the rows may not be running the same harness at all.

12.5. Work backwards from the decision. If you would switch models for a 5-point improvement but not a 2-point one, you need about 280 examples per model, which is affordable. If you would switch for 1 point, you need about 6,900, which usually is not — and the correct response is to stop trying to resolve it and choose on cost, latency and operational fit. That is not a failure of rigour; it is the rigorous conclusion.

12.17 Best Practices

BEST PRACTICE

Reading scale and reading benchmarks

1. **Plot a continuous metric beside every all-or-nothing one.** It costs one line and it is the cheapest protection against believing in a threshold that is not there.
2. **State `n` beside every score**, always, including internally.
3. **Compute the sample size before quoting a gap.** A 2-point difference at 90% needs about 1,700 examples per model.
4. **Allow "not established" as a conclusion** and use it. Most small gaps are.
5. **Never extrapolate a fit beyond a small multiple of the range you fitted**, and be more suspicious the fewer seeds you ran.
6. **Report R^2 with any fitted exponent.** A power law will fit anything if you do not check.
7. **Hold the evaluation set fixed** across every model in a comparison, seeded identically.
8. **Sample your evaluation set from your real distribution, including the tail**, because behaviour at the edges is discontinuous.
9. **Choose the metric before you see any scores**, and choose it to match the decision the output feeds.
10. **Keep your selection benchmark private**, or it will eventually be trained on.
11. **Check for contamination** by comparing performance on public benchmarks against structurally identical private ones.
12. **Assume scaffolding differences** on any agentic comparison until shown otherwise.
13. **Treat parameter counts as memory, and training budgets as quality** — and note that neither is published for Claude.
14. **Convert "let's wait for the next model" into an estimate with an error bar**, including the estimate that it will not be enough.

12.18 Common Mistakes

COMMON MISTAKE

Reading a metric artefact as an emergent capability

Symptom. A capability appears to switch on between two model sizes, and a roadmap is built on the assumption that the next size unlocks something further.

Cause. The metric is all-or-nothing over several parts, so it is a smooth curve raised to a power. Our exact-match score crossed its range in 2.4× of parameters while per-digit accuracy on the same eleven checkpoints took 16.6×.

Fix. Plot a continuous score — loss, per-part accuracy, log-likelihood of the correct answer — on the same checkpoints. If the sharpness survives, investigate. It usually does not.

Detection. Compute `transition_range(k)` for your task's number of parts. If the band is narrow, expect an apparent threshold regardless of what the model is doing.

COMMON MISTAKE

Quoting a two-point benchmark gap as a reason

Symptom. A model-selection document cites a small benchmark difference, the chosen model underperforms in production, and nobody can reconstruct why the decision was made.

Cause. No sample size was checked. Around 90%, resolving two points takes roughly 1,700 examples per model, and most benchmarks are smaller than that.

Fix. Run `diff_established`. If the interval crosses zero, write "not established" and choose on cost, latency and your own evaluation.

Detection. Any comparison document with percentages and no `n`.

COMMON MISTAKE

Extrapolating a scaling fit far past its range

Symptom. A projection of what the next model generation will achieve turns out badly wrong, in either direction.

Cause. The exponent is a slope estimated from a segment, and slopes from short noisy segments are unreliable. Our seven-model fit gave `b = 0.417`; the full sweep gave `b = 0.957`, and the short fit mispredicted the largest model's loss by a factor of ten.

Fix. Fit across as wide a range as you can afford, with multiple seeds, and treat predictions beyond a small multiple of that range as directional only.

Detection. Compare your fitting range against your prediction range. If the ratio is large, you are guessing with decimal places.

COMMON MISTAKE

Comparing models on different evaluation sets or scaffolding

Symptom. An internal comparison contradicts a later, more careful one, or fails to reproduce.

Cause. The eval set was regenerated between runs, or the prompt, retry policy or tool set differed. The comparison then measures the harness, not the model.

Fix. Seed the evaluation set identically and hold every non-model variable fixed. There is a test for this in the mini project precisely because the failure is silent.

Detection. Re-run the losing model with the winner's exact harness. If the gap closes, the harness was the finding.

COMMON MISTAKE

Treating a scaling law as a promise about your task

Symptom. More compute or a larger model is procured on the strength of a published curve, and the specific capability that motivated it does not improve.

Cause. Scaling laws describe average next-token loss over a broad distribution. Your task is a thin slice measured with a metric you chose, and the relationship is loose.

Fix. Fit the trend on **your** task if you can, and otherwise treat published laws as a prior rather than a forecast.

Detection. Ask what quantity the cited curve actually plots. If it is loss and your requirement is a task success rate, they are different things.

COMMON MISTAKE

Using parameter count as the quality axis

Symptom. A comparison concludes that a larger model must be better, or that a small model cannot possibly be sufficient.

Cause. Chinchilla. Capacity is set by parameters; how much of it is used is set by the training budget. A 70B model on 15T tokens and a 70B model on 300B tokens are a well-trained model and an under-trained one.

Fix. Compare on measured performance. Chapter 9 gives you the memory arithmetic that parameter counts *are* good for.

Detection. Any argument of the form "it is a larger model, so".

12.19 Performance Considerations

A scaling sweep is embarrassingly parallel and should be run that way. Every size is independent, so the wall-clock cost of a sweep is the cost of the largest model, not the sum. Our sweep was written to be resumable — it appends to a JSON file and skips sizes already present — because it was run inside short sessions, and that same design makes it trivially distributable across machines.

Cost grows faster than parameters. By `C ≈ 6ND`, doubling parameters at a fixed token budget doubles training compute, and doing it properly under Chinchilla means doubling tokens too, which quadruples it. This is why sweeps are done at small scale and extrapolated, and why section 12.6's warning about

extrapolation is a practical constraint rather than a pedantic one.

Evaluation cost is what actually limits a selection study. Establishing a 1-point difference needs about 6,900 examples per model. On a frontier model with long outputs, at Chapter 11's rates, that is a real invoice — and it is exactly the workload the Batch API's 50% discount exists for. Section 12.16's advice to stop trying to resolve very small gaps is a cost argument as much as a statistical one.

Prefer cheap continuous metrics for tracking, expensive discrete ones for deciding. Loss and per-part accuracy give you a usable signal from far fewer examples than an all-or-nothing rate does, because they carry more information per item. Track the continuous metric to see the trend; measure the discrete one, properly sized, when a decision depends on it.

12.20 Security Considerations

A public benchmark is a training target. Anything you publish will eventually appear in training data, at which point it measures memorisation. This applies to your own evaluation sets: if you put them in a public repository, a blog post or a support ticket, treat their lifespan as finite.

Benchmark scores are a claim surface. Vendors, internal teams and consultants all have incentives to present the most flattering configuration — the best prompt, the strongest harness, the most favourable subset. This is not usually fraud; it is selection. Reproduce any number that will change a significant decision, with your own harness.

Contamination has a safety dimension, not only a quality one. A model that scores well on a safety benchmark it has memorised is not safer. Chapter 8's layered architecture is the reason this is survivable: safety evaluations inform how much you rely on the model, and never replace the controls that do not depend on it.

Metric choice can conceal a regression. A change that improves a partial-credit score while degrading exact-match will look like an improvement on the wrong dashboard, and exact-match is usually the one your users experience. Report both, and let the discrete one be the one that gates a release.

12.21 Real Production Examples

The roadmap built on a knee. The chapter's opening incident: a team saw an internal exact-match score leap 4.7× for a 1.4× increase in model size and planned a quarter's work on the assumption that the next increment would unlock a further capability. Plotting per-digit accuracy on the same checkpoints took twenty minutes and showed an unremarkable straight line. The generalisable control is a standing rule — every all-or-nothing chart is published with a continuous companion.

The two-point migration. A team migrated a production workload to a different model on the strength of a 2.2-point benchmark advantage, and observed no measurable improvement in their own metrics. The benchmark had a few hundred items; establishing 2.2 points at that accuracy needed roughly 1,700 per model. The migration was not wrong, exactly — it was unsupported, and the effort would have been better spent on the evaluation set that would have settled it.

The small model that was better. A team assumed a frontier model was required for a classification task and never tested the cheapest tier, on the reasoning that it had far fewer parameters. When they

finally measured, the gap on their task was under a point and unresolved at their sample size, while Chapter 11's rate card put the cost difference at 5×. Chinchilla-era intuitions about small models being weak had aged badly, exactly as section 12.4 predicts.

The harness that was the finding. An agentic benchmark comparison showed one model well ahead. Reproducing it with a common harness — same prompt, same tools, same retry policy — closed most of the gap. The published rows had used different scaffolding. Nobody had misreported anything; the table simply did not measure what it appeared to.

12.22 Certification Notes

Concept in this chapter	CCAR-F domain	How it is examined
Training budget dominates parameter count	D1 Agentic Architecture and Orchestration	Distractor: "choose the model with more parameters"
Emergence as a metric artefact	D5 Context Management and Reliability	Scenario: a capability appears to switch on
Sample size for a reported difference	D5	Distractor: a small gap from a small benchmark
Benchmarks do not select models; your evaluation does	D1, D5	Scenario: model selection with a leaderboard in the stem
All-or-nothing versus partial-credit metrics	D5	Which metric matches the decision the output feeds
Contamination and saturation	D5	Why an old benchmark's high score is weak evidence
Scaffolding differences	D1, D2 Tool Design and MCP Integration	Why an agentic comparison may measure the harness
Inference-optimal versus training-optimal	D1	Why small modern models are strong, and cheap
Scaling predicts loss, not capability	D1	Distractor: "the next model will fix this"
Not established as a valid conclusion	D5	Recurs; the mark is for saying so rather than guessing

CERTIFICATION NOTE

What the exam does with benchmarks

Benchmark questions almost never ask you to know a score. They ask you to notice that a score is being over-read.

"Team X chose model A because it scored 2 points higher." The expected answer identifies that the difference is not established at the benchmark's sample size, and that selection should rest on an evaluation of the team's own task. An option offering to run that evaluation is nearly always correct; an option defending the choice by citing the benchmark is nearly always a distractor.

"Capability Y appeared between model sizes." Look for the all-or-nothing metric in the stem. The expected answer separates the metric from the model.

"Should we wait for a better model?" This is Chapter 10's degree-or-kind question with scaling attached. If the failure is structural — exact arithmetic, statelessness, character-level work — waiting buys nothing and the answer is architecture. If it is a matter of degree, waiting is legitimate, and the strong answer says how you would estimate what it buys.

A recurring trap: options that justify a model choice by parameter count. Chinchilla is the reason those are wrong, and Claude's counts are not published in any case.

12.23 Practice Questions

1. A scaling law relates training resources to loss with which functional form? A) Linear B) Exponential C) Power law D) Logistic

2. Plotted on log-log axes, a power law appears as: A) A curve B) A straight line C) A step D) A parabola

3. The compute identity `C ≈ 6ND` says training FLOPs are proportional to: A) Parameters only B) Tokens only C) Parameters × tokens D) Parameters squared

4. Chinchilla's central finding was that, for a fixed training budget: A) Parameters should grow much faster than data B) Parameters and data should scale roughly in proportion C) Data alone matters D) Architecture dominates both

5. Chinchilla's compute-optimal ratio is approximately how many training tokens per parameter? A) 1 B) 5 C) 20 D) 200

6. Modern models such as Llama 3.1 are trained well past the compute-optimal ratio because: A) Chinchilla was wrong B) Inference is paid on every request, so a smaller model is permanently cheaper to serve C) Data is free D) It improves the parameter count

7. In this chapter's sweep, per-digit accuracy crossed its range over a 16.6× parameter span while exact-match crossed in 2.4×. This shows: A) A genuine phase transition B) That the metric, not the model, produces the apparent threshold C) A bug in the sweep D) That exact-match is more accurate

8. For a task whose answer has `k` parts that must all be right, the aggregate score is approximately: A) `per_part × k` B) `per_part ÷ k` C) `per_part ^ k` D) `per_part`

9. At 50 parts, the per-part accuracy range that takes an all-or-nothing score from 10% to 90% is roughly: A) 0.10 to 0.90 B) 0.50 to 0.95 C) 0.955 to 0.998 D) 0.99 to 1.00

10. Around a 90% accuracy rate, establishing a 2-point difference requires approximately how many examples per model? A) 100 B) 300 C) 1,700 D) 50,000

11. Halving the difference you wish to resolve changes the required sample size by a factor of: A) 2 B) 4 C) 8 D) 1

12. Fitting this chapter's power law on the seven smallest models and predicting the largest: A) Was accurate B) Over-predicted loss by up to 10× C) Under-predicted loss D) Could not be computed

13. Benchmark contamination means: A) Corrupted data files B) Benchmark items appearing in training data C) Too many examples D) Mixed languages

14. Two models score 96.1% and 95.8% on the same benchmark. The most likely explanation is: A) The first is meaningfully better B) Saturation — the gap is within the benchmark's own label noise C) Contamination D) A scaffolding difference

15. On agentic benchmarks, the largest confound is often: A) Temperature B) Scaffolding differences between rows C) Context length D) Tokenizer version

16. A published capability index reported as "162.1, 95% CI 158.0–167.3, n = 40" is good practice because: A) The number is large B) It gives an interval and a sample size C) It uses 40 benchmarks D) It is a round number

17. Scaling laws predict which of the following well? A) Whether a specific capability works B) Instruction-following reliability C) Loss on the training distribution D) Refusal behaviour

18. The correct conclusion when a comparison's interval crosses zero is: A) The models are identical B) Not established C) The larger model wins D) Increase temperature

19. Why is a parameter count a poor quality signal? A) It is hard to measure B) Capacity is set by parameters but how much is used is set by the training budget C) It changes over time D) It is proprietary

20. The best benchmark for deciding which model to deploy is: A) The largest public leaderboard B) The most recent one C) Your own task, distribution and metric D) The one the vendor cites

12.24 Answers and Explanations

1 — C. Power law, almost always with an irreducible floor term. **B** is a common misreading of a log-scaled chart.

2 — B. Which is what makes the law extendable, and why extrapolation is tempting.

3 — C. Roughly `6ND`: about `2N` FLOPs per token forward, twice that backward, times `D` tokens. The consequence is that `N` and `D` trade directly at fixed compute.

4 — B. Roughly proportional scaling, correcting Kaplan's earlier parameter-heavy recommendation. **A** is the pre-2022 consensus, which is why the correction mattered.

5 — C. About 20 tokens per parameter.

6 — B. Training is paid once; inference is paid forever and scales with size. Over-training a smaller model buys permanently cheaper serving, which is Chapter 11's economics reaching back into the training decision.

7 — B. Same eleven checkpoints, two metrics, one of which is the other raised to the seventh power. **A** is the conclusion the introduction's team reached.

8 — C. `per_part ^ k` when parts fail independently. The approximation degrades when errors correlate, which the chapter measures rather than assumes.

9 — C. 0.9550 to 0.9979, a span of 0.043. At fifty parts almost any smooth improvement crossing that band looks like a breakthrough.

10 — C. About 1,729 per model. Most benchmarks are smaller, which is why most two-point gaps are unresolved.

11 — B. Four, because sample size scales with `1/delta²`. This is the fact that should end most attempts to resolve small differences.

12 — B. Over-predicted loss by up to 10.6× at the largest model — that is, the real models were *better* than the short fit forecast. A 9× fitting range does not license a 59× prediction.

13 — B. Benchmark items or close paraphrases in the training data, which converts memorisation into an apparent capability.

14 — B. Saturation. Above roughly 95% you are largely comparing how two models handle the benchmark's own mislabelled items.

15 — B. Prompt, tools, retries and harness can differ more between published rows than the models do, and on agentic tasks the harness is often worth more than the model choice.

16 — B. An interval and a sample size — precisely what section 12.10's fifth failure is the absence of. It is the reporting standard to copy internally.

17 — C. Loss on the training distribution, and aggregate trends. **A**, **B** and **D** are training outcomes measured with metrics of your choosing, and the relationship to loss is loose.

18 — B. Not established. **A** is a distinct and stronger claim that the data does not support either.

19 — B. Chinchilla. Two models with identical dimensions and different data budgets are a well-trained model and an under-trained one.

20 — C. Your own task, distribution and metric, sized for the difference that would change your decision. A hundred well-chosen examples from your own traffic beats any leaderboard.

12.25 Ten Practical Exercises

1. Run the sweep and reproduce the two transition widths.
2. Compute `transition_range(k)` for your own task's number of must-be-right parts.
3. Take your last three model-comparison documents and run `diff_established` on every claim in them.
4. Add `n` and an interval to every score your evaluation harness prints.
5. Compute the sample size needed for the smallest difference that would change a decision you face this quarter.

6. Build a private held-out set from your production traffic, including the awkward tail, and keep it out of any repository that leaves your organisation.
7. Re-score one existing evaluation with a partial-credit metric alongside the all-or-nothing one, and compare the shapes.
8. Estimate the training-token-to-parameter ratio of two open-weight models and place them relative to Chinchilla-optimal.
9. Reproduce a published benchmark row with your own harness and report the gap between your number and theirs.
10. Write the paragraph you would send a team that wants to plan a quarter around an apparent capability threshold.

12.26 Five Challenge Labs

1. **A proper scaling study.** Run the sweep with five seeds per size across at least two orders of magnitude of parameters, fit with confidence intervals on the exponent, and report how much of the single-seed scatter was variance.
2. **Data scaling.** Hold parameters fixed and vary the training set size instead. Fit the data-scaling exponent and compare it against the parameter-scaling exponent, then say what your result implies about the compute-optimal allocation for this task.
3. **Manufacture an emergence.** Take a genuinely smooth capability and design a metric that makes it look like a phase transition, then a second metric that makes the same data look smooth. Write the two accompanying paragraphs a press release would use.
4. **Contamination probe.** Design an experiment that would detect whether a benchmark you rely on is contaminated, using structurally identical items the model cannot have seen. State what result would convince you either way.
5. **Selection study, end to end.** Choose between two models for a real workload: size the study, build the private set, run it, report the interval, and make the decision — including the decision to choose on cost because the difference was not established.

12.27 Interview Questions

1. What is a scaling law, and what does it actually predict?
2. Explain `C ≈ 6ND` and what it implies for a fixed training budget.
3. What did Chinchilla change, and why did the field accept it so quickly?
4. Why are modern models trained far past the compute-optimal token ratio?
5. A capability appears to switch on between two model sizes. How would you investigate?
6. Why does an all-or-nothing metric make smooth improvement look like a threshold?
7. How many examples do you need to establish a 2-point difference at 90% accuracy, and why?
8. Name three ways a benchmark table can mislead, with a check for each.
9. Why is parameter count a poor proxy for quality?
10. How would you decide between two models for a workload you own?

12.28 Summary

Scaling laws are empirical power laws relating training resources to loss, and their practical value is that a power law is a straight line on log-log axes, so it can be extended. The compute identity $C \approx 6ND$ makes them actionable by turning a budget into a choice between parameters and data, and Chinchilla settled that choice: scale them roughly in proportion, at around twenty training tokens per parameter. A 70-billion-parameter model trained on 1.4 trillion tokens beat a 280-billion-parameter model trained on 300 billion at the same compute — which is the direct answer to Chapter 9's assertion that training budget dominates parameter count, and the deepest reason a parameter count tells you about memory rather than quality.

Serving changes the question. Training is paid once and inference is paid forever, so the deployment-oriented recipe is to take a smaller model and train it far past compute-optimal, buying permanently cheaper inference with one-off training compute. Llama 3.1's 8B model saw roughly ninety times more data per parameter than Chinchilla advises, and that is deliberate. It also means intuitions about what a small model can do age badly, which is why Chapter 15 insists on measuring rather than assuming — and why the tiers on Chapter 11's rate card are points on a training-versus-serving curve rather than simply "big, medium, small".

Fitting a law ourselves was instructive in both directions. Eleven models across a 59× parameter range produced clean power laws in loss and per-digit error, with R^2 of 0.907 and 0.878 — real trends, honestly short of the 0.99-plus that careful multi-seed work reports, because we ran one seed per size. And extrapolating from the seven smallest models mispredicted the largest model's loss by a factor of ten, in the optimistic direction. The exponent is a slope, slopes estimated from short noisy segments are unreliable, and the exponent is exactly what compounds across an extrapolation. Scaling laws predict an average trend within the regime you fitted; they do not predict your task, your metric, or the specific run you are about to launch.

The chapter's central result concerned none of that. Across the same eleven checkpoints, loss and per-digit accuracy took a 16.6× parameter range to cross their spans, while exact-match accuracy took 2.4× and looked exactly like a switch being thrown. Nothing discontinuous happened inside the models. An answer has seven digits and counts only if all seven are right, so the sharp curve is the smooth curve raised to the seventh power — an approximation we verified to within 0.007 above 0.9 per-digit accuracy, and which degrades to 0.087 below it, where digit errors correlate. Generalising, at fifty must-be-right parts the entire visible transition from useless to working occupies a per-part accuracy band from 0.955 to 0.998. Any smooth improvement crossing that band presents as a breakthrough. The defensible claim is not that emergence never happens; it is that a sharp curve is not evidence of a qualitative change until you have plotted a continuous metric on the same checkpoints.

Which leaves benchmarks, and the same discipline applied to somebody else's table. Contamination turns memorisation into apparent capability; saturation turns label noise into a ranking; scaffolding differences turn a harness comparison into a model comparison; prompt sensitivity turns a distribution into its upper tail; and missing intervals turn noise into a decision. That last one has a number attached: establishing a two-point gap around 90% takes about 1,700 examples per model, and halving the gap you want to resolve quadruples that. Most published gaps of a point or two are consistent with no difference at all. Good reporting looks like the Opus 5 system card's capability index — a point estimate, a 95%

interval, and a stated `n` — and that is the standard to hold your own reports to. Then build the only benchmark that decides anything: your task, your distribution, your metric, chosen before you see any scores, sized for the difference that would change your mind, and kept private. A hundred examples of your own beat any leaderboard, provided you are willing to print the interval and say "not established" when it is.

KEY TAKEAWAYS

1. **Scaling laws are power laws in loss** — straight lines on log-log axes, empirical rather than derived.
2. `C ≈ 6ND`: at fixed compute, parameters and tokens trade directly.
3. **Chinchilla**: scale them in proportion, ~20 tokens per parameter. Training budget dominates parameter count.
4. **Inference-optimal ≠ training-optimal.** Modern small models are deliberately over-trained, some by 90×.
5. **Our fit gave R^2 of 0.907** — and extrapolating from a 9× range to a 59× one was wrong by 10×.
6. **Emergence is usually the metric.** 16.6× versus 2.4× transition width on the same eleven checkpoints.
7. **All-or-nothing scores are per-part accuracy raised to the number of parts.** At 50 parts the knee is 0.955–0.998.
8. **A 2-point gap at 90% needs ~1,700 examples per model**, and halving the gap quadruples that.
9. **Five checks before a table changes a decision**: contamination, saturation, scaffolding, prompt, intervals.
10. **Select on your own task**, with your metric chosen first, your interval printed, and "not established" allowed.

12.29 Glossary

Term	Definition
All-or-nothing metric	A score awarding credit only when every part of an answer is correct.
Chinchilla	The 2022 result that parameters and training tokens should scale roughly in proportion, ~20:1.
Compute-optimal	The parameter/data allocation minimising loss for a fixed training budget.
Contamination	Benchmark items present in training data, converting memorisation into apparent capability.
Emergent capability	Performance that appears to switch on at some scale; frequently an artefact of the metric.
Established difference	A gap whose confidence interval excludes zero.
Irreducible loss	The floor term `c`; the entropy of the data that no model can beat.
Over-training	Training past the compute-optimal token ratio to buy cheaper inference.
Power law	`y = a·x^(−b)`; a straight line in log-log space.
Saturation	Scores near the ceiling, where a benchmark stops distinguishing models.

Term	Definition
Scaffolding	The prompt, tools, retries and harness surrounding a model in a benchmark run.
Scaling exponent	The `b` in a power law; governs how fast loss falls, and the fragile part of any fit.
System card	A published report of a model's capability and safety evaluations.
Transition width	The resource range over which a metric crosses 10% to 90% of its span.

12.30 Further Reading

Primary sources. Kaplan et al., *Scaling Laws for Neural Language Models* (2020) established the power-law framework; Hoffmann et al., *Training Compute-Optimal Large Language Models* (2022) — the Chinchilla paper — corrected the parameter/data allocation and is the more important of the two for practitioners. Schaeffer, Miranda and Koyejo, *Are Emergent Abilities of Large Language Models a Mirage?* (2023) is the careful statement of section 12.7's argument, and is worth reading in full because it is more nuanced than its title. Wei et al., *Emergent Abilities of Large Language Models* (2022) is the paper it responds to; read both. For reporting practice, Anthropic's system cards — the Claude Opus 5 card dated 24 July 2026 and the Claude Sonnet 5 card dated 30 June 2026 — show what capability and safety evaluation looks like when it is done with intervals and stated sample sizes, including third-party red-teaming.

Where to go next in this book. Chapter 13 covers reasoning models and test-time compute — the most significant recent change in what more compute buys, and the one place where the scaling axis moved from training to inference. Chapter 14 introduces the Claude model family concretely and Chapter 15 turns everything in this chapter into a selection procedure with Chapter 11's costs on the other side of the ledger. Chapter 24 builds the regression suites that this chapter's statistics belong in, and Chapter 57 covers the observability and evaluation harnesses that keep them running. Chapter 62 maps the certification blueprint chapter by chapter, and Chapters 63 and 64 are where you will find out whether the discipline stuck.